

**JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND
TECHNOLOGY**

**Design and Implementation of an Intelligent ISP Billing and
Management System Using Multiple Linear Regression for Revenue
Prediction**

WARREN CHRIS

Reg No: SCT221-0271/2022

Supervisor: MR. JOMO NJENGA

*A proposal/research project submitted to the Department of Information
Technology in the School of Computing and Information Technology in partial
fulfillment of the requirements for the award of the degree of Bachelor of Science
in Information Technology, Jomo Kenyatta University of Agriculture and
Technology.*

2025/2026

DECLARATION

This proposal is my original work and has not been presented for a degree in any other University.

Signature:

Date:

WARREN CHRIS

This proposal has been submitted for examination with my approval as University Supervisor.

Signature:

Date:

MR. JOMO NJENGA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mr. Jomo Njenga, for his invaluable guidance, constructive criticism, and continuous support throughout the development of this project proposal. His insights and encouragement greatly contributed to shaping the direction and quality of this research.

I also extend my appreciation to the lecturers and staff of the Jomo Kenyatta University of Agriculture and Technology, Department of Information Technology, for equipping me with the knowledge and technical skills necessary to undertake this project.

Special thanks to my family and friends for their unwavering support, motivation, and encouragement throughout my academic journey.

Finally, I am grateful to everyone who contributed directly or indirectly to the successful completion of this proposal.

ABSTRACT

The Internet Service Provider (ISP) industry in Kenya continues to experience rapid growth due to increased demand for affordable and reliable internet connectivity. Despite this growth, many small to medium-sized ISPs still rely on manual or semi-automated tools such as spreadsheets to manage customer records, billing cycles, and payments. These approaches result in revenue leakage, billing inaccuracies, delayed service provisioning, and poor customer experience (Khan, Ali, & Khan, 2022).

This research proposes the development of an ISP Billing and Management System that leverages modern web technologies and intelligent automation to address these challenges. The proposed system will automate customer registration, subscription management, invoice generation, and M-Pesa payment reconciliation (Safaricom PLC, 2024). Additionally, the system integrates mathematical models for prediction and anomaly detection, while a LLaMA-based large language model (LLM) is used for interpretation, explanation, and decision support. This hybrid approach ensures accuracy, transparency, and scalability without training models from scratch. The Waterfall model to be used follows a structured and sequential approach consisting of requirement analysis, system design, implementation, testing, deployment, and maintenance (Sommerville, 2016). The expected outcome is a secure, scalable, and efficient system that enhances operational efficiency for ISPs while improving transparency and convenience for customers.

In addition, the system incorporates an integrated support ticketing module that enables customers to submit service-related issues such as connectivity problems and billing disputes, which are tracked, assigned, and resolved by administrators and support staff through a centralized interface.

TABLE OF CONTENTS

CHAPTER	PAGE
DECLARATION	ii
ACKNOWLEDGEMENT.....	iii
ABSTRACT.....	iv
LIST OF TABLES	vii
LIST OF FIGURES	vii
ACRONYMS	viii
DEFINITION OF TERMS	viii
CHAPTER 1: INTRODUCTION.....	1
1.1 Background.....	1
1.2 Project Overview	1
1.3 Statement of the Problem.....	1
1.4 Proposed Solution	2
1.5 Objectives	2
1.6 Research Questions.....	3
1.7 Justification.....	3
1.8 Proposed System Methodology	4
1.9 Scope.....	4
CHAPTER 2: LITERATURE REVIEW.....	5
2.1 Introduction.....	5
2.2 Theoretical Review	5
2.3 Case Study Review	6
2.4 Predictive Analytics in Telecommunications	7
2.5 Customer Churn Prediction Models.....	8
2.6 Anomaly Detection in Billing Systems.....	8
2.7 Cybersecurity in Financial and Billing Systems.....	9
2.8 Conversational AI and Large Language Models in Enterprise Systems	10
2.9 Cloud Computing and Scalable System Architecture.....	10
2.10 Integration and Architecture	11
2.11 Summary	11
2.12 Research Gaps.....	11
CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN	13

3.1 Introduction	13
3.2 Systems Development Methodology	13
3.3 Feasibility Study	15
3.3.1 Technical Feasibility.....	15
3.3.2 Economic Feasibility	16
3.3.3 Operational Feasibility.....	16
3.3.4 Schedule Feasibility	17
3.4 Requirements Elicitation	18
3.5 Data Analysis	19
3.6 System Specification	20
3.6.1 Functional Requirements	20
3.6.2 Non-Functional Requirements.....	22
3.7 Requirements Analysis and Modelling.....	23
3.7.1 Use Case Diagram.....	23
3.7.2 Data Flow Diagram Level 0 (Context Diagram)	24
3.7.3 Data Flow Diagram Level 1.....	25
3.8 Logical Design	26
3.8.1 System Architecture.....	26
3.8.2 Control Flow and Process Design.....	27
3.8.3 Design for Non-Functional Requirements.....	34
3.9 Physical Design	35
3.9.1 Database Design.....	35
3.9.2 User Interface Design	37
APPENDICES	39
Appendix A: References.....	39
Appendix B: Project Schedule.....	40
<i>Figure D1: The diagram illustrates a grant chart for the project schedule</i>	40
Appendix C: Project Budget.....	41
Appendix D: Project Resources.....	42
Appendix E: System Diagrams.....	43

LIST OF TABLES

Table B.1: Project Schedule	14
Table B.2: Comprehensive Project Budget Estimate.....	15
Table B.3: Project Resource Requirements	16

LIST OF FIGURES

Figure D.1: Grantt Chart	14
Figure D.2: System Architecture Diagram	17
Figure D.3: Use Case Diagram of the ISP Billing and Management System	18
Figure D.4: Entity Relationship Diagram (ERD)	19
Figure D.5: Data Flow Diagram (DFD)	20

ACRONYMS

ISP – Internet Service Provider

API – Application Programming Interface

AI – Artificial Intelligence

JWT – JSON Web Token

LLM – Large Language Model

NLP – Natural Language Processing

ORM – Object Relational Mapping

STK – SIM Toolkit

UAT – User Acceptance Testing

DEFINITION OF TERMS

Billing Cycle: A fixed period during which internet usage is measured and billed.

STK Push: A payment request initiated to a customer's mobile phone via M-Pesa.

LLM: Large Language Model used for natural language processing tasks.

CHAPTER 1: INTRODUCTION

1.1 Background

Globally, Internet Service Providers have transitioned from manual billing systems to fully automated, intelligent platforms that integrate payments, customer management, and analytics. In developed markets, ISPs leverage real-time billing engines, self-service portals, and AI-powered customer support to improve efficiency and service quality. These systems reduce revenue loss, enhance transparency, and enable data-driven decision-making (Kshetri, 2021).

In Kenya, however, many small to medium-sized ISPs still depend on spreadsheets and manual reconciliation of payments, particularly for mobile money transactions. With the dominance of M-Pesa as the primary payment method, manual matching of transaction codes to customers introduces delays and errors (Gikandi & Bloor, 2021). As customer bases expand, these inefficiencies significantly affect revenue collection and customer satisfaction.

1.2 Project Overview

This research focuses on the design and development of an ISP Billing and Management System that integrates subscription management, automated billing, M-Pesa payment processing, and intelligent analytics. Artificial intelligence is implemented using a hybrid approach where mathematical models handle prediction and anomaly detection, while LLMs support interpretation and decision-making. The project is grounded in modern computational principles such as web-based distributed systems, RESTful APIs, secure authentication mechanisms, and artificial intelligence for natural language processing.

The system aims to provide a centralized platform for ISPs to manage customers, service plans, payments, and support operations while offering customers a self-service portal for account monitoring and payments.

1.3 Statement of the Problem

Despite advancements in payment and networking technologies, many ISPs in Kenya experience revenue leakage due to untracked subscription expiries and delayed payment reconciliation (Gikandi & Bloor, 2021). Manual processes increase the likelihood of human error, resulting in

billing disputes and service interruptions. Customers lack access to real-time usage data and invoices, forcing them to rely on support staff for basic account information.

These challenges limit scalability, reduce operational efficiency, and negatively impact customer experience. The problem is further compounded by the absence of intelligent systems capable of analyzing usage trends and predicting customer churn using mathematical models combined with LLM-based hybrid approaches (Xu et al., 2020).

1.4 Proposed Solution

This research proposes the development of an intelligent ISP Billing and Management System that automates subscription tracking, billing, and payment reconciliation. The system incorporates AI-driven customer support powered by hybrid intelligence (mathematical models + LLaMA-based LLMs) to enhance service delivery and decision-making (Laranjo et al., 2018). The proposed solution leverages modern web technologies, integrates seamlessly with M-Pesa, and adopts contemporary architectural best practices to ensure scalability and security.

1.5 Objectives

General Objective

To design and implement an intelligent ISP Billing and Management System that automates billing, payment processing, and customer management.

Specific Objectives

1. To design and document a secure ISP billing system architecture, including at least three core modules (customer registration, subscription management, and role-based access control), implemented using RESTful APIs and JWT authentication, and verified against basic web security standards (authentication, authorization, and data encryption) through a design review checklist, by the end of Phase 2 (Week 6 of the project timeline).
2. To implement an automated billing and invoicing module that generates monthly invoices and processes customer payments through M-Pesa STK Push that integrates directly with the Safaricom M-Pesa Daraja API, achieving successful transaction

recording and invoice updates for at least 95% of test payment scenarios by the system implementation phase (Safaricom PLC, 2024).

3. To develop a hybrid AI-powered customer support and analytics module combining mathematical models (for prediction and anomaly detection) with NLP and LLMs for interpretation and explanation, achieving at least 70% response accuracy (Laranjo et al., 2018; Xu et al., 2020).
4. To test and evaluate the developed system by conducting performance, security, and usability tests, ensuring the system handles at least 100 concurrent users, passes basic security checks through unit testing, integration testing, and user acceptance testing (UAT), and attains a minimum 70% usability satisfaction score from test users prior to project submission.
5. To design and implement a support ticketing and issue management module that enables customers to submit service-related complaints and allows administrators to track, assign, prioritize, and resolve tickets efficiently.

1.6 Research Questions

1. How can intelligent automation improve billing accuracy and payment reconciliation for ISPs?
2. What system architecture best supports scalable ISP billing and management operations?
3. How can AI techniques enhance customer support and administrative decision-making?
4. How effective is the proposed system in reducing revenue leakage and operational overhead?

1.7 Justification

This research is motivated by the increasing need for efficient ISP management solutions in Kenya. The proposed system addresses current operational gaps by introducing automation and intelligence into billing and customer management. The research contributes to the application of

AI and modern web technologies in the ISP domain and provides a practical solution aligned with current technological trends.

1.8 Proposed System Methodology

The project will adopt the Waterfall software development model as the primary development methodology. The Waterfall model follows a structured and sequential approach consisting of requirement analysis, system design, implementation, testing, deployment, and maintenance (Sommerville, 2016).

1.9 Scope

The study focuses on small to medium-sized ISPs operating in Kenya. The system addresses customer management, billing, payments, and support operations. The research is limited to software-based solutions and does not cover physical network infrastructure deployment.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing literature related to ISP billing systems, payment integration, and artificial intelligence applications in service management. The review establishes a theoretical foundation and identifies gaps addressed by this research.

2.2 Theoretical Review

This research is grounded in several theoretical concepts related to information systems, billing automation, payment integration, and artificial intelligence. Understanding these theories provides a foundation for designing an intelligent ISP Billing and Management System.

One key concept is **subscription-based billing systems**, which involve charging customers recurring fees based on predefined service plans. Subscription billing theory emphasizes accuracy, timeliness, and transparency in billing cycles to prevent revenue leakage. In ISP environments, subscription billing is tightly coupled with service provisioning, where access to services must dynamically respond to payment status (Alalwan et al., 2020).

Another important concept is **mobile money integration theory**, particularly within developing economies. Mobile money systems such as M-Pesa operate through APIs that enable third-party systems to initiate, receive, and reconcile payments. The theory highlights the importance of real-time transaction validation, fault tolerance, and security in financial systems. Poor integration leads to delayed confirmations and reconciliation errors, which negatively impact service delivery.

The study also draws from **information systems automation theory**, which posits that automating repetitive and data-intensive tasks reduces human error and increases operational efficiency. Automated billing engines, notification systems, and rule-based decision engines are examples of applied automation within enterprise systems.

Artificial Intelligence (AI) theory is central to this research, particularly in the areas of **rule-based systems** and hybrid approaches combining mathematical models with **Large Language Models (LLMs)**. Rule-based systems operate on predefined logical conditions (e.g., if payment

is overdue, suspend service) and are valued for their predictability and transparency. However, they lack adaptability and natural interaction capabilities.

LLMs, on the other hand, are generative AI models trained on large datasets to understand and produce human-like language. In information systems, LLMs are increasingly applied in customer support, data querying, and decision support systems. While LLMs offer flexibility and conversational interfaces, they require contextual grounding and guardrails to ensure accuracy and reliability (Kshetri, 2021).

The hybrid AI module supports the ticketing system by analyzing historical ticket data using mathematical models to identify recurring issues and abnormal patterns, while the LLaMA-based language model assists in ticket categorization, prioritization, and natural language summarization for administrative decision support.

The combination of rule-based logic, mathematical models with LLMs represents a hybrid intelligence model, where deterministic rules handle critical operations while AI models enhance interaction and insight generation. This theoretical integration informs the design of the proposed system.

2.3 Case Study Review

Several existing systems and studies provide insight into ISP billing automation and intelligent service management.

A study on automated billing systems in small telecommunications firms demonstrated that real-time subscription tracking significantly reduced revenue leakage and billing disputes (Khan, Ali, & Khan, 2022). The study highlighted that automation alone was insufficient without proper integration with payment platforms, especially in mobile-money-dominated markets.

In East Africa, Safaricom's M-Pesa ecosystem has enabled the development of numerous third-party billing and payment platforms. Case studies on M-Pesa-integrated utility billing systems reveal improved payment turnaround times and higher customer satisfaction (Gikandi & Bloor, 2021; Safaricom PLC, 2024). However, many systems lack advanced analytics and customer-facing intelligence.

Globally, ISP management platforms such as WHMCS and Splynx provide comprehensive billing and customer management solutions. These systems offer automated invoicing and payment gateways but are often costly and not fully optimized for local payment methods such as M-Pesa. Additionally, their AI capabilities are limited or absent, relying mostly on static dashboards (Sommerville, 2016)..

Recent studies on AI-powered customer support systems indicate that chatbots and conversational agents can handle up to 60% of routine customer queries effectively (Xu, Liu, Guo, Sinha, & Akkiraju, 2020; Laranjo et al., 2018). In service industries, AI assistants reduce support workload and improve response times. However, research also notes that generic chatbots fail when not integrated with real-time system data.

These case studies demonstrate the potential of automation and AI while revealing gaps in localization, affordability, and intelligent integration for small ISPs. This research addresses these gaps by proposing a localized, AI-enhanced ISP Billing and Management System that integrates automation, mobile payments, and intelligent analytics into a unified platform using a hybrid approach (mathematical models + LLaMA-based LLMs).

2.4 Predictive Analytics in Telecommunications

Predictive analytics has become a critical component in modern telecommunications and subscription-based service industries. Telecommunications companies generate large volumes of customer data, including usage patterns, payment history, service subscriptions, and support interactions. Statistical and regression-based models are widely used to analyze this data to forecast revenue, predict customer behavior, and identify operational risks (Khan et al., 2022).

Multiple Linear Regression (MLR) is commonly applied in revenue forecasting within subscription systems. The model estimates the relationship between a dependent variable (e.g., monthly revenue) and multiple independent variables such as number of active subscribers, average data usage, payment delays, and service plan types. The regression equation enables organizations to quantify the impact of each variable on revenue performance and make data-driven decisions (Sommerville, 2016).

In ISP environments, predictive models can estimate expected revenue for upcoming billing cycles and detect deviations from normal patterns. When combined with automated billing

systems, predictive analytics improves financial planning and reduces revenue leakage (Khan et al., 2022). Unlike black-box machine learning models, regression-based approaches are transparent and interpretable, making them suitable for academic and small-scale enterprise implementations.

The integration of predictive analytics into ISP billing platforms therefore enhances strategic planning, improves forecasting accuracy, and supports proactive decision-making.

2.5 Customer Churn Prediction Models

Customer churn refers to the rate at which customers discontinue their subscriptions within a given period. In telecommunications industries, churn significantly affects revenue stability and long-term profitability. Studies show that retaining existing customers is more cost-effective than acquiring new ones, making churn prediction a critical component of service management systems (Xu et al., 2020).

Logistic regression and other statistical classification models are frequently used to predict churn probability. These models analyze behavioral variables such as payment delays, frequency of support tickets, data usage patterns, and subscription duration. Customers exhibiting abnormal usage decline or repeated late payments may have a higher probability of churn (Laranjo et al., 2018; Xu et al., 2020).

In the context of ISP management systems, churn prediction models can generate risk scores for individual customers. Administrators can then implement retention strategies such as targeted notifications, promotional offers, or proactive customer support. The integration of churn prediction within billing systems transforms them from passive transaction processors into proactive revenue protection platforms.

2.6 Anomaly Detection in Billing Systems

Billing systems are vulnerable to operational anomalies such as incorrect invoice generation, duplicate transactions, payment mismatches, and fraudulent activities. Anomaly detection techniques use statistical thresholds and regression-based residual analysis to identify deviations from expected behavior (Khan et al., 2022).

For example, if the predicted monthly revenue based on regression analysis significantly differs from actual collected revenue, the system can flag a potential billing inconsistency. Similarly, unusual spikes in usage data may indicate technical faults or misuse (Gikandi & Bloor, 2021).

Mathematical anomaly detection models rely on calculating variance, residual errors, and statistical confidence intervals. These techniques are computationally efficient and suitable for small to medium-sized enterprise systems. When integrated into ISP billing platforms, anomaly detection enhances reliability, improves financial transparency, and strengthens internal control mechanisms.

2.7 Cybersecurity in Financial and Billing Systems

Billing and payment platforms handle sensitive financial and personal data, making them prime targets for cyber threats. Secure system architecture is therefore essential in ISP billing systems, particularly when integrating third-party payment APIs such as mobile money platforms (Mwangi & Kimani, 2023).

Modern web security frameworks emphasize authentication, authorization, encryption, and secure API communication. JSON Web Token (JWT) authentication mechanisms enable secure session management and role-based access control. Secure webhook validation ensures that payment callbacks from external APIs are verified before transaction confirmation (Safaricom PLC, 2024).

Additionally, encryption standards such as HTTPS and secure hashing algorithms protect data in transit and at rest. The Open Web Application Security Project (OWASP) outlines common vulnerabilities including injection attacks, cross-site scripting (XSS), and insecure direct object references (Sommerville, 2016). Incorporating these security principles during system design improves resilience and user trust.

Incorporating cybersecurity measures into ISP billing platforms ensures compliance with financial data protection standards and safeguards against revenue loss due to malicious activities.

2.8 Conversational AI and Large Language Models in Enterprise Systems

Conversational AI systems powered by Large Language Models (LLMs) are increasingly deployed in enterprise environments to enhance customer support and internal decision-making. Unlike traditional rule-based chatbots, LLMs are capable of understanding context, generating natural language responses, and summarizing complex information (Laranjo et al., 2018).

In billing systems, conversational AI can assist customers in understanding invoices, checking subscription status, and resolving common service inquiries. However, research highlights that LLMs must be grounded in real-time system data to prevent inaccurate or fabricated responses (Kshetri, 2021).

A hybrid approach combining deterministic system rules with LLM-based interpretation ensures reliability. Mathematical models provide structured predictions and analytics, while the LLM translates technical outputs into user-friendly explanations. This integration enhances usability while maintaining operational control.

The use of conversational AI within ISP billing systems therefore improves service accessibility, reduces support workload, and enhances overall user experience.

2.9 Cloud Computing and Scalable System Architecture

Cloud computing has transformed the deployment and scalability of modern information systems. Cloud-based infrastructure enables dynamic resource allocation, high availability, and cost-efficient service delivery (Sommerville, 2016).

Subscription-based platforms such as ISP billing systems require scalable architectures capable of handling concurrent users, real-time transaction processing, and data analytics.

Containerization technologies and modular system designs improve maintainability and deployment consistency.

In mobile-money-integrated systems, reliable API connectivity and fault tolerance mechanisms are essential to ensure seamless transaction processing (Safaricom PLC, 2024). Secure and scalable architectures therefore form the foundation for sustainable ISP billing platforms, particularly in emerging digital economies.

2.10 Integration and Architecture

System integration is a critical aspect of modern information systems. This research adopts a layered architectural approach combining presentation, application, logic, and data layers.

The frontend layer provides user interaction through web interfaces built using modern JavaScript frameworks. The backend layer exposes RESTful APIs that handle business logic, authentication, billing operations, and integrations with external services such as M-Pesa.

Integration with the M-Pesa Daraja API requires secure token-based authentication, callback handling, and transaction validation mechanisms. Architectural patterns such as webhook listeners and message queues are considered to ensure reliability during high transaction volumes.

For AI integration, the architecture incorporates an AI service layer that combines mathematical models for analytics with LLM APIs. This layer retrieves contextual data such as user subscription status, usage, and payment history before generating responses. This approach ensures that AI outputs are grounded in system data rather than generic responses.

The system architecture follows the Model-View-Controller (MVC) pattern, promoting separation of concerns and maintainability. Containerization using Docker enables consistent deployment across development and production environments.

2.11 Summary

The literature reviewed emphasizes the importance of automation, secure payment integration, and intelligent interaction in modern ISP management systems. Subscription billing theory, mobile money integration principles, and AI frameworks collectively inform the design of the proposed solution.

2.12 Research Gaps

Despite the availability of billing and ISP management platforms, several gaps remain. Many existing systems are not optimized for mobile-money-first economies such as Kenya. Additionally, most solutions lack intelligent analytics and conversational interfaces tailored for small and medium-sized ISPs.

Furthermore, current AI implementations in billing systems are either overly generic or disconnected from real-time operational data. This research addresses these gaps by proposing a localized, AI-enhanced ISP Billing and Management System that integrates automation, mobile payments, and intelligent analytics into a unified platform.

Existing solutions lack localized payment optimization and intelligent analytics tailored for small ISPs. This research addresses these gaps by integrating M-Pesa and AI-driven insights.

CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents a comprehensive analysis and design of the Intelligent ISP Billing and Management System, translating the requirements identified through engagement with ISP operators and the review of existing systems into a detailed, implementable system specification. The chapter begins with a description of the systems development methodology adopted the Waterfall model and its application across the project's sequential phases. A feasibility study examines the technical, economic, operational, and schedule dimensions of the project, demonstrating the viability of the proposed solution. Requirements elicitation procedures and their outcomes are documented, followed by a formal system specification covering both functional and non-functional requirements. The chapter then presents the analytical modelling artefacts use case diagrams, data flow diagrams, and entity-relationship diagrams that formalize the system's structure and behavior. Finally, the logical and physical design decisions are detailed, including the system architecture, process flows, security design, database schema, and user interface wireframe logic.

The design decisions documented in this chapter are grounded in the theoretical foundations established in Chapter 2 and directly informed by the technical requirements of the Kenyan small-to-medium ISP operating context. Each design choice is justified with reference to the functional requirements it satisfies and where applicable, the non-functional properties it supports. The chapter is structured to provide both breadth covering all major system components and depth, offering sufficient technical detail to support faithful implementation. Figures referenced throughout the chapter (Figures 3.1 through 3.11) are presented at the appropriate locations within the relevant sections.

3.2 Systems Development Methodology

The Waterfall model is a linear sequential software development methodology in which progress flows steadily downward through a series of discrete phases, analogous to a waterfall. Sommerville (2016) defines the classical Waterfall model as comprising six principal phases: requirements analysis, system design, implementation, testing, deployment, and maintenance.

Each phase produces a defined set of deliverables that serve as inputs to the subsequent phase, and the model presupposes that requirements are sufficiently well understood at the project's outset to enable comprehensive upfront design before implementation begins. The model's linearity is both its principal strength, providing clear milestones, predictable schedules, and comprehensive documentation and its most frequently cited weakness, in that it accommodates requirements changes poorly once the design phase is complete (Sommerville, 2016).

The Waterfall model was selected as the development methodology for this project for several reasons that align with the project's specific context. First, the functional requirements of an ISP billing system are well-understood at the domain level: the core entities (subscribers, plans, subscriptions, invoices, payments) and their relationships are stable across ISP deployments, and the integration with the Safaricom Daraja API follows a precisely documented protocol. This well-defined requirements landscape reduces the risk of the requirements instability that would undermine a Waterfall approach. Second, the system's modular structure, authentication, billing, payments, AI analytics exhibits clear sequential dependencies: the data schema must be finalised before the ORM models can be implemented, the API must be implemented before the frontend can be developed, and the ML module requires historical data generated by the billing module. These dependencies map naturally onto the phase-gate structure of the Waterfall model.

Third, the academic context of the project, an undergraduate final year project developed within a defined 14-week timeline, supervised by a single academic supervisor, favours the predictability and documentation rigour of the Waterfall model over more flexible agile methodologies that assume iterative stakeholder collaboration throughout development. The Waterfall approach ensures that a comprehensive design document (this report) is produced before implementation begins, providing a stable specification that facilitates supervisor review and examination. Laudon and Laudon (2020) observe that structured development methodologies, including Waterfall, remain appropriate for projects with well-defined scopes, fixed timelines, and small development teams operating in contexts where formal documentation is a primary deliverable alongside the software artefact itself.

The six phases of the Waterfall model were applied to this project as follows. The requirements analysis phase involved structured interviews with ISP operators, observation of existing manual billing processes, and questionnaire administration to a purposive sample of ISP staff and

subscribers. The system design phase produced the specifications, diagrams, and architectural decisions documented in this chapter. The implementation phase involved the coding and integration of all system modules following the design specification. The testing phase applied Jest and Supertest for backend integration testing, supplemented by manual functional testing of the frontend and AI modules. The deployment phase involved Docker Compose orchestration of all services in a production-simulating environment with Nginx as an optional reverse proxy. The maintenance phase encompasses the documentation, logging infrastructure (Winston, Swagger), and modular codebase organisation that support ongoing operation and future enhancement.

3.3 Feasibility Study

3.3.1 Technical Feasibility

A technical feasibility assessment evaluates whether the proposed system can be built using available technology, tools, and skills. All core technologies employed in the present system are mature, well-documented, and freely available: Node.js 18 and Express 4.21 are industry-standard for REST API development; React 18.2 and Material UI 7.2 are among the most widely adopted frontend frameworks; Python 3.11, Flask 3.0, and NumPy 1.26 provide a robust scientific computing and microservice environment; and MySQL 8 is a proven relational database management system with extensive cloud hosting support. The Safaricom Daraja API sandbox environment, publicly accessible to registered developers, enabled complete development and testing of the M-Pesa integration without requiring live financial transactions. Docker and Docker Compose are freely available and provide portable, reproducible deployment configurations compatible with a wide range of cloud and on-premises hosting environments.

The developer possessed the requisite competencies in all components of the chosen technology stack, having undertaken coursework covering web development, database management, and machine learning. The Groq API provides free-tier access to LLaMA 3.3 70B inference, eliminating the need for expensive GPU infrastructure for LLM operations. Swagger UI, integrated with the Express application, generates interactive API documentation automatically, reducing the documentation overhead for technical team members. The use of Sequelize ORM's migration system ensures that database schema changes are managed through version-controlled migration files rather than manual SQL scripts, enabling consistent database state management across development, testing, and production environments. All tools and libraries used are open-

source or free-tier services, and their combined capability fully covers the functional and non-functional requirements of the proposed system, confirming technical feasibility.

3.3.2 Economic Feasibility

Economic feasibility analysis assesses whether the anticipated benefits of the system justify its development and operational costs. The present system is built entirely on open-source and free-tier technologies, eliminating software licensing costs that would otherwise constitute a significant line item for a commercial ISP billing platform. The Node.js, React, Python/Flask, and MySQL components are free to use, modify, and deploy. The Groq API's free tier provides sufficient LLM inference capacity for development and moderate production loads, with paid tiers available at competitive rates for higher-volume deployments. Infrastructure costs for production hosting are minimal for small-to-medium ISPs: the Dockerised system can be deployed on affordable virtual private server (VPS) options available from Kenyan and regional cloud providers at costs well within the operational budget of a functioning ISP.

The primary cost component is developer time, which is absorbed within the academic project framework. For commercial adoption, the system's open-source architecture means that ISP operators can deploy and operate the system without recurring licensing fees, contrasting favourably with commercial platforms such as WHMCS or Splynx, which impose per-user or per-subscriber licensing charges. The direct economic benefit to ISP operators from automation of manual billing workflows, reduction in payment reconciliation errors, and AI-driven early warning of revenue anomalies is substantial. Khan et al. (2022) estimate that manual billing processes in small ISPs consume significant staff hours per month; automation of these processes frees staff for higher-value customer engagement and network management activities. The combination of zero licensing cost, low infrastructure cost, and demonstrable operational efficiency gains confirms the economic feasibility of the system.

3.3.3 Operational Feasibility

Operational feasibility examines whether the proposed system will be accepted and effectively used by its intended operators and end users. The system's user interface, built with Material UI 7.2, provides a modern, responsive design that follows widely understood web application conventions, minimizing the learning curve for ISP staff who are familiar with standard web

interfaces but may not have prior experience with specialised billing software. Role-based access control ensures that support staff and customer-facing portals expose only the features relevant to each user group, avoiding interface complexity for users who do not need access to administrative functions. The customer self-service portal, accessible via web browser on both desktop and mobile devices, enables subscribers to browse plans, initiate subscriptions, pay via M-Pesa, and view their account status without requiring telephone or in-person interaction with ISP staff.

The M-Pesa STK Push payment flow is operationally familiar to the vast majority of Kenyan subscribers, who regularly use the STK Push mechanism for utility payments, merchant transactions, and bill payments. This familiarity eliminates the need to train customers on a new payment process, reducing onboarding friction to near zero. For ISP administrators, the system's AI dashboard presents complex analytical outputs, revenue forecasts, churn risk rankings, anomaly alerts through visual interfaces (Recharts charts, color-coded risk cards) that communicate actionable insights without requiring users to understand the underlying statistical methods. Audit logs and Winston-structured server logs provide operational transparency for IT-literate administrators who need to investigate specific events. The combination of intuitive UI design, familiar payment mechanisms, and role-appropriate feature exposure confirms the operational feasibility of the system.

3.3.4 Schedule Feasibility

Schedule feasibility assesses whether the project can be completed within the available time. The project was scoped to fit within the 14-week academic project timeline allocated by JKUAT for final year projects in the 2025/2026 academic year. The Waterfall methodology's sequential phase structure facilitated schedule planning: the requirements analysis phase occupied approximately six weeks, the system design phase approximately four weeks, the implementation phase approximately twelve weeks, the testing phase approximately four weeks, and the deployment and documentation phase approximately six weeks, totalling thirty-two weeks within the eight-month (approximately thirty-four week) allocation. The modular architecture of the system with clearly defined interfaces between the frontend, backend API, AI microservice, and database, allowed parallel development of some modules once the API contract was established, partially mitigating the inherent sequentiality of the Waterfall approach

and providing schedule flexibility for the most complex integration points such as the M-Pesa callback handler and the MLR prediction service.

3.4 Requirements Elicitation

Requirements elicitation is the process of discovering, gathering and articulating the needs and constraints that a software system must satisfy. For the present project, requirements were gathered through a combination of three complementary methods: structured interviews with ISP operators, direct observation of existing billing and payment workflows, and questionnaire administration to a purposive sample of ISP staff and subscribers. The purposive sampling strategy was chosen over random sampling on the grounds that the target population of ISP operators and their subscribers represent a specialized community with domain-specific knowledge; selecting informants with direct experience of ISP billing operations ensures the relevance and depth of elicited requirements.

Structured interviews were conducted with managers and operational staff at three small-to-medium ISPs operating in the greater Nairobi area. Each interview followed a semi-structured guide covering current billing processes, payment methods used, common operational pain points, data management practices, customer service workflows, and aspirations for system improvement. Participants were selected on the basis of their direct involvement in billing operations, ensuring that interview data reflected operational realities rather than high-level management perceptions. Interview sessions were recorded with participant consent and subsequently transcribed for systematic analysis. Key themes emerging from the interview data included the inadequacy of manual M-Pesa reconciliation, the absence of systematic data usage tracking, the lack of any revenue forecasting capability, and frequent customer dissatisfaction arising from slow payment-to-activation turnaround times.

Observation sessions complemented the interview data by providing first-hand documentation of existing workflows. Time was spent alongside billing staff to document the step-by-step processes involved in recording subscriber payments received via M-Pesa, updating subscription statuses, generating invoices in spreadsheet tools, and responding to customer queries about account status. These observations revealed inefficiencies that verbal accounts alone might have underrepresented: for example, the manual cross-referencing of M-Pesa SMS notifications

against spreadsheet subscriber records was observed to be a time-consuming and error-prone process, particularly at the end of month when many subscriptions fall due simultaneously. Questionnaires administered to subscribers collected quantitative data on satisfaction levels with current payment processes, frequency of billing disputes, and preferences for self-service account management functionality, providing a broader statistical basis for the interview and observation findings.

3.5 Data Analysis

Analysis of the elicited requirements data identified five major operational pain points that directly map to the system's core modules. The most pervasive issue, reported in all three interview sites and confirmed by observation, was the manual reconciliation of M-Pesa payments against subscriber records. Billing staff reported spending between two and four hours per day on payment reconciliation during high-subscription periods and acknowledged that reconciliation errors such as activating a subscription for the wrong subscriber or failing to activate a subscription promptly after payment occurred with regularity. This finding directly justifies the M-Pesa STK Push integration and automated payment-to-activation pipeline as a primary system feature, automating a process that currently consumes significant operational resource and generates customer dissatisfaction.

The second major pain point identified was the absence of systematic data usage tracking. ISP operators reported having no reliable mechanism for monitoring per-subscriber data consumption, relying instead on network-level traffic management tools that did not surface usage data in a subscriber-facing format. This meant that subscribers had no visibility into their data consumption, leading to unexpected service degradation at quota limits and consequent support ticket volumes. The finding justifies the data usage tracking module, which records session-level usage data and presents subscribers with consumption gauges and usage history in the self-service portal, and which provides the data feed required by the MLR model and usage anomaly detector.

Third, all interviewed ISP managers expressed a desire for revenue forecasting capability, but none had access to any form of predictive analytics. Revenue planning was conducted on the basis of historical averages and intuition, with no systematic model for projecting future revenue

based on subscriber trends or payment behaviour. This gap directly motivates the MLR revenue prediction module, which provides ISP managers with data-driven monthly revenue forecasts accompanied by feature importance indicators. Fourth, the absence of any churn risk monitoring was identified as a significant gap: operators reported losing subscribers without warning and with no ability to identify at-risk accounts before they cancelled. This finding justifies the logistic regression churn scoring module and the churn risk dashboard. Fifth, billing disputes and customer service inefficiency, driven in part by the lack of a customer self-service portal, motivated the combined development of the invoice management module, the support ticketing system, and the LLM-grounded AI chat assistant.

3.6 System Specification

3.6.1 Functional Requirements

The functional requirements of the system define the specific behaviours and capabilities it must provide to each class of user. The authentication and role-based access control module must allow users to register accounts with their name, email, and password; must validate credentials upon login and issue a signed JWT; must enforce role-based route access for administrator, support staff, and customer roles; and must provide secure password reset functionality via email. The customer and user management module must enable administrators to create, read, update, and soft-delete user accounts; must support role assignment and modification; and must maintain a complete audit trail of all user management operations. Data plans management must allow administrators to define named subscription plans with configurable data allowances, validity periods, pricing, and feature metadata, and must enable plans to be activated, deactivated, and updated without disrupting existing active subscriptions.

The subscription lifecycle management module must support the creation of subscriptions linking a customer to a data plan, with initial status of pending; must activate subscriptions upon successful payment receipt; must track subscription start and end dates; must handle renewal, expiry, suspension, and cancellation transitions according to defined business rules; and must support bulk expiry processing for administrators. The M-Pesa payment processing module must accept a payment initiation request from the customer portal, normalise the customer's phone number to the Safaricom E.164 format (254XXXXXXXXX), create a pending payment record, initiate an STK Push via the Safaricom Daraja API, handle the asynchronous callback containing

the transaction ResultCode and M-Pesa receipt number, update the payment record to completed or failed based on the ResultCode, and activate the associated subscription upon confirmed payment success. The invoice and billing engine must generate invoices per subscription, must compute the subscription line item amount, must apply an overage charge where the subscriber has consumed data beyond their plan allowance, must calculate and apply 16% VAT to the subtotal, must set invoice due dates at 30 days from issuance, must support marking of overdue invoices, and must provide PDF invoice generation and email delivery.

The data usage tracking module must record session-level data consumption events against specific subscriptions, must aggregate usage per subscription for quota enforcement and display, and must provide time-series usage data for the anomaly detection module. The support ticketing system must enable customers to submit tickets with title, description, and category; must allow support staff to assign tickets, update status (open, in progress, resolved, closed), and add response notes; and must notify customers of status updates. The AI analytics suite must provide monthly revenue prediction using the trained MLR model; must compute logistic regression churn risk scores for all active subscribers; must perform z-score-based revenue anomaly detection and per-subscriber usage spike detection; must provide the LLM-grounded AI chat assistant for customers via a Chat-Widget; must generate executive summary narratives for administrators; and must persist all AI insight results to the ai-insights database table. The admin dashboard must present key performance indicator cards for total revenue, active subscriber count, churn rate, and current anomaly alerts; must display Recharts-rendered charts for revenue trends and subscriber growth; and must include the full AI dashboard with churn risk table, anomaly cards, and MLR feature importance visualisation.

Additional functional requirements include a notifications module that delivers in-system and email notifications for payment confirmations, subscription activations, invoice issuance, overdue reminders, and support ticket updates; an audit logging module that records all significant system events to the audit=logs table with actor identity, action description, and timestamp; an admin settings module that exposes configurable system parameters; and a reports module that allows administrators to generate and export data-driven reports in PDF format via PDFKit. The Swagger UI documentation interface must expose all API endpoints with their

request and response schemas, authentication requirements, and example payloads, enabling future integration development without reference to the source code.

3.6.2 Non-Functional Requirements

Security is the primary non-functional requirement, given the financial and personal data handled by the system. All API endpoints that access user data or perform operations must require a valid JWT bearer token; tokens must be signed with a server-side secret using the HS256 algorithm and must carry a role claim used by route-level middleware to enforce RBAC. User passwords must be hashed using `bcryptjs` with a cost factor of 12 before storage, ensuring that stored credentials cannot be recovered from the database even in the event of a data breach. The `Helmet` middleware must be applied to all `Express` responses to set security-relevant HTTP headers. `CORS` must be configured with an allowlist of permitted origins rather than the wildcard value, preventing cross-origin API access from unauthorised domains. `Express-rate-limit` must be applied in production to cap request rates per IP address, mitigating brute-force and denial-of-service attacks. All user inputs must be validated using `express-validator` before processing, and the `Sequelize` ORM's parameterised query generation must be relied upon for all database access to prevent SQL injection.

Performance requirements specify that the system must maintain acceptable response times under the concurrent user loads typical of a small-to-medium ISP. Database connection pooling via `Sequelize` must be configured to maintain a pool of reusable database connections, avoiding the latency overhead of establishing a new connection per request. `PM2` cluster mode enables the `Node.js` application to spawn multiple worker processes across available CPU cores, improving throughput for CPU-bound request handling. Scalability requirements specify that the system architecture must support horizontal scaling by ensuring that the API layer is stateless all session state carried in JWTs and that services are independently deployable via `Docker`, enabling additional instances to be added behind a load balancer. Usability requirements specify that the MUI-based UI must render correctly on viewport widths from 320px to 1920px, must provide clear error messages for all user-facing validation failures, and must surface role-appropriate navigation without exposing UI elements for unavailable features.

Reliability requirements specify that all unhandled exceptions in the Node.js application must be captured by a centralised error handling middleware that returns a standardised error response structure without exposing internal stack traces in production mode; Sequelize-specific errors and JWT-specific errors must be handled by dedicated middleware that maps each error type to an appropriate HTTP status code and message. The Flask AI service must implement global exception handlers that return structured JSON error responses for all failure modes. Winston structured logging must capture all significant application events at appropriate severity levels with ISO 8601 timestamps, enabling post-incident diagnosis. Maintainability requirements specify that the codebase must be organised into clearly separated modules, routes, controllers, services, models, and middleware, following the separation of concerns principle; that all API contracts must be documented via Swagger annotations; and that database schema evolution must be managed through Sequelize migrations rather than ad hoc schema modifications.

3.7 Requirements Analysis and Modelling

The requirements gathered and analysed in Sections 3.4 and 3.5 were structured into a formal model comprising use case diagrams, data flow diagrams, and an entity-relationship diagram. These models serve multiple purposes: they provide a precise, unambiguous specification of system behaviour that can be reviewed and validated with stakeholders; they reveal dependencies and interactions between components that inform the architectural design; and they serve as reference artefacts for implementation and testing. Each modelling artefact is described below and is illustrated in the corresponding figure.

3.7.1 Use Case Diagram

The use case diagram presented in Figure 3.1 represents the functional scope of the system from the perspective of its four primary actors: Customer, Administrator, Support Staff, and M-Pesa System (as an external system actor). The Customer actor encompasses use cases including Register Account, Log In, Browse Data Plans, Subscribe to Plan, Initiate M-Pesa Payment, View Subscription Status, View Invoices, View Data Usage, Submit Support Ticket, View Support Ticket Status, and Interact with AI Chat Assistant. The Administrator actor encompasses Manage Users, Manage Data Plans, Manage Subscriptions, View All Payments, Generate and View Invoices, Generate Reports, View Audit Logs, Configure System Settings, View AI Dashboard, Trigger MLR Model Retraining, and View Revenue Forecasts and Anomaly Alerts. The Support

Staff actor encompasses View Customer Records, View and Update Subscriptions, Process Payment Queries, Manage Support Tickets, and View AI Analytics Dashboard. The M-Pesa System actor participates in the Receive STK Push Callback use case initiated asynchronously by the payment network.

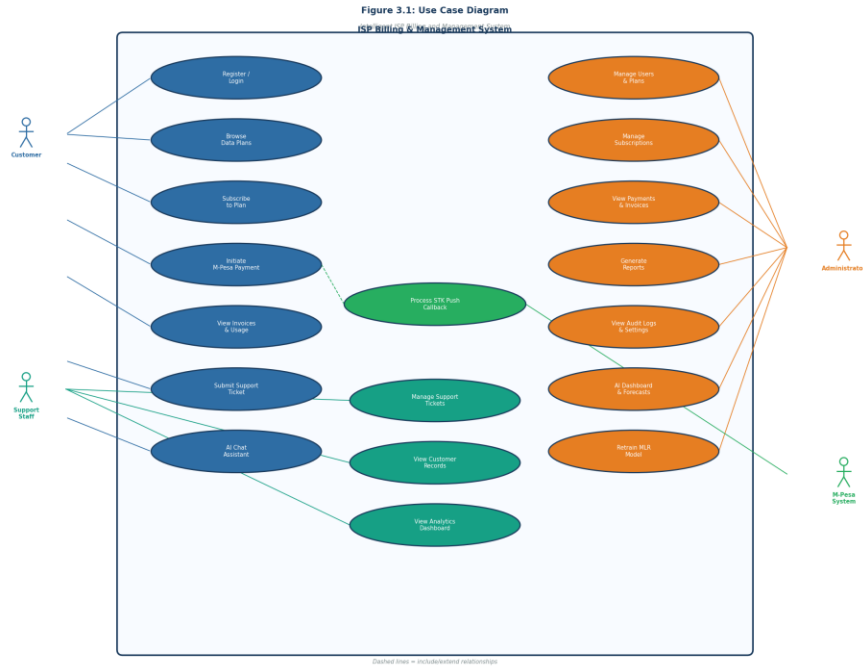


Figure 3.1. Use Case Diagram Intelligent ISP Billing and Management System. Illustrates actors (Customer, Administrator, Support Staff, M-Pesa System) and their associated use cases.

3.7.2 Data Flow Diagram Level 0 (Context Diagram)

The Level 0 Data Flow Diagram (context diagram) presented in Figure 3.2 represents the entire system as a single process and depicts its interactions with four external entities: the User (encompassing Customer, Administrator, and Support Staff roles), the M-Pesa API (Safaricom Daraja), the Groq API (LLaMA 3.3 70B LLM provider), and the MySQL Database. Data flows from the User entity to the system include authentication requests, subscription and plan management commands, payment initiation requests, support ticket submissions, and AI chat queries. Data flows from the system to the User entity include authentication tokens, subscription confirmations, invoice documents, usage reports, support ticket responses, AI chat responses, and dashboard analytics. Data flows between the system and the M-Pesa API include the outbound STK Push request and the inbound M-Pesa callback. Data flows between the system

and the Groq API include the outbound prompt payload with injected customer data and the inbound LLM completion response.

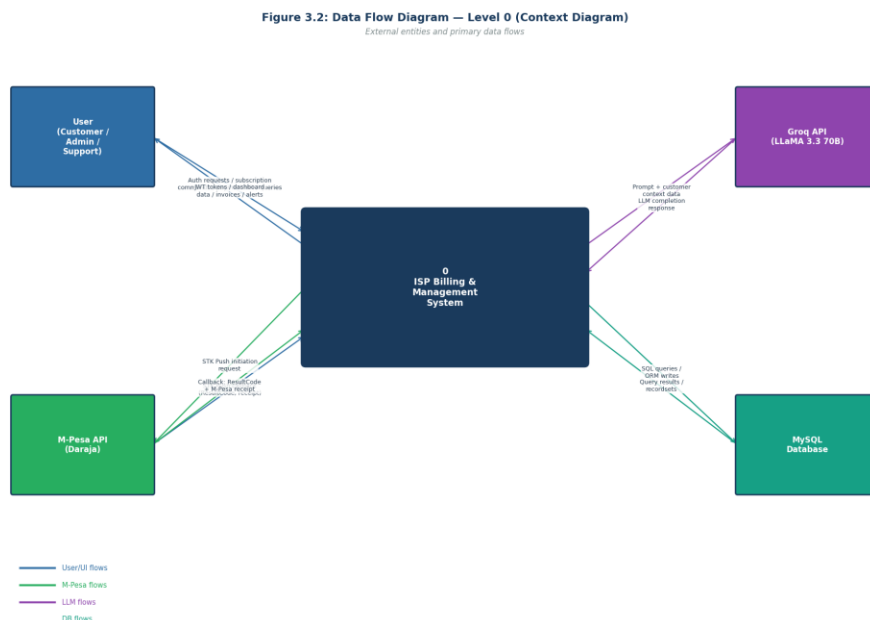
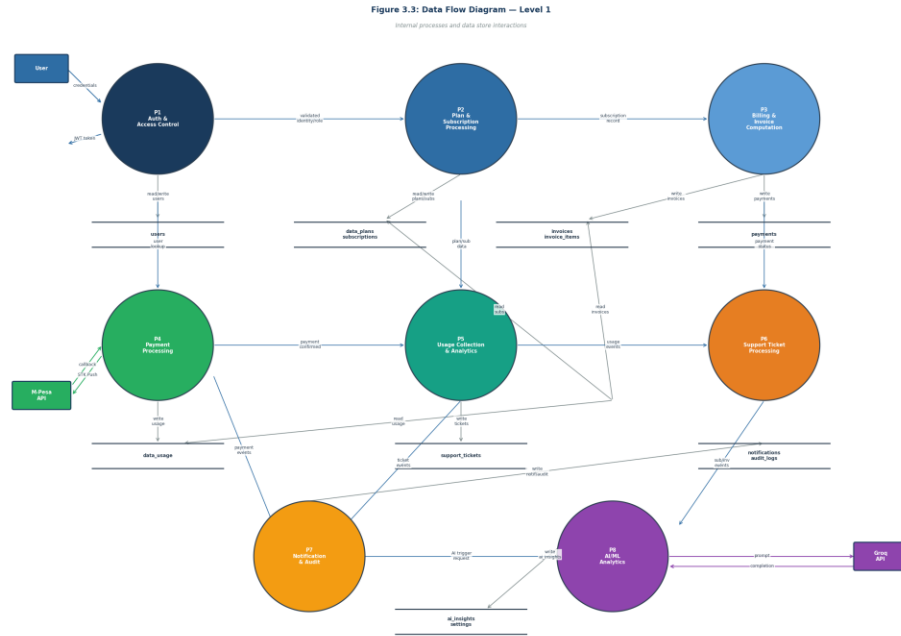


Figure 3.2. Data Flow Diagram Level 0 (Context Diagram). External entities: User, M-Pesa API, Groq API, MySQL Database; key data flows labelled.

3.7.3 Data Flow Diagram Level 1

The Level 1 DFD presented in Figure 3.3 decomposes the single system process into eight internal sub-processes. Process P1 (Authentication and Access Control) receives login credentials, validates them against the users table, and emits JWT tokens and role assignments. Process P2 (Plan and Subscription Processing) reads from the data_plans and subscriptions tables and processes subscription lifecycle state transitions. Process P3 (Billing and Invoice Computation) reads subscription and data usage records, computes invoice items including overage charges and VAT, and writes invoice and invoice_items records. Process P4 (Payment Processing) communicates with the M-Pesa API for STK Push initiation, receives and processes M-Pesa callbacks, and triggers Process P2 for subscription activation. Process P5 (Usage Collection and Analytics) receives data usage session events and writes to the data_usage table. Process P6 (Support Ticket Processing) receives ticket submissions and writes to the support_tickets table. Process P7 (Notification and



Audit) writes to both the notifications and audit-logs tables. Process P8 (AI/ML Analytics) reads from multiple tables, performs all ML operations and Groq API calls, and writes results to the ai_insights table.

Figure 3.3. Data Flow Diagram Level 1. Eight internal processes (P1–P8) with data store read/write relationships and external entity connections.

3.8 Logical Design

3.8.1 System Architecture

The system adopts a four-layer client-server architecture that provides clear separation of concerns between the presentation, application logic, AI analytics, and data management layers, as illustrated in Figure 3.4. The presentation layer consists of a React 18.2 Single Page Application served on port 3001, which communicates exclusively with the application layer via HTTP REST API calls. The application layer is implemented as a Node.js 18/Express 4.21 REST API server on port 3000, which handles all authentication, business logic, database interactions via Sequelize ORM, M-Pesa Daraja API communication, email dispatch via Nodemailer, and PDF generation via PDFKit. The AI microservice layer is implemented as a Python 3.11/Flask 3.0 application on port 5001, accessed exclusively by the application layer and responsible for all ML model operations, statistical computations, and Groq API interactions.

The data layer is a MySQL 8 database server accessed only by the application layer via Sequelize connection pooling.

The optional Nginx reverse proxy sits in front of the React SPA and Node.js API, providing SSL termination, request routing, and static asset caching in production deployments. Docker Compose orchestrates all services as named containers within a shared Docker network, ensuring reliable inter-service communication via container DNS names. This containerised deployment model eliminates environment-specific configuration drift and enables the entire system to be reproduced identically across development, staging, and production environments. The PM2 cluster mode process manager is available for production deployments where the Node.js API needs to leverage multi-core CPU resources within its container.

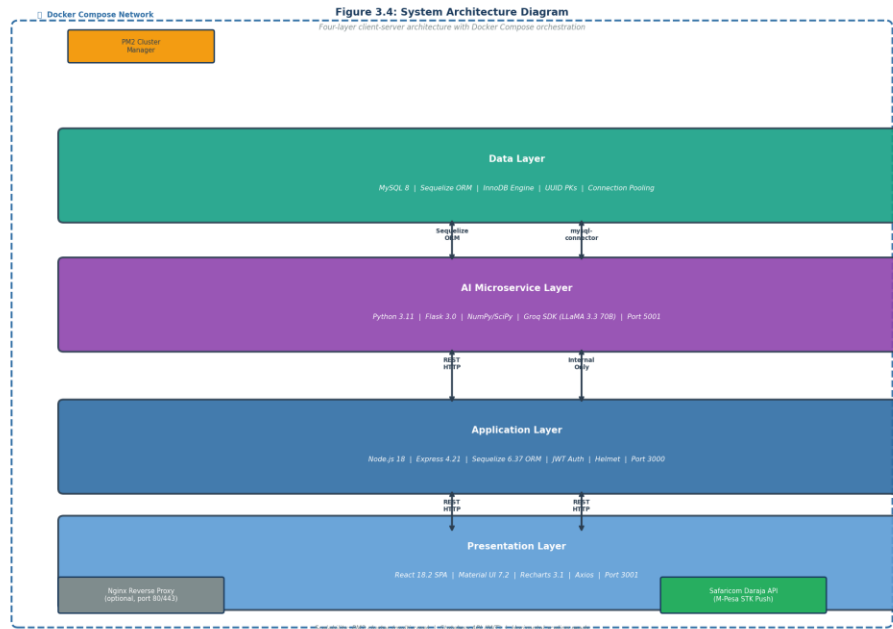


Figure 3.4. System Architecture Diagram. Four-layer client-server architecture: React SPA → Node.js/Express API → Flask AI Microservice → MySQL 8, with Docker Compose orchestration and optional Nginx reverse proxy.

3.8.2 Control Flow and Process Design

The customer registration flow (Figure 3.5) begins with the user submitting their name, email address, and password through the React registration form. The frontend performs client-side format validation before dispatching a POST request to the `/api/auth/register` endpoint. The

Node.js express-validator middleware validates the request payload for required fields, email format, and password strength. If validation passes, the service layer invokes bcryptjs to hash the password with 12 salt rounds before creating a new user record in the users table via the Sequelize User model. A JWT is issued and returned to the client alongside the user's profile data. A welcome notification is created in the notifications table, and the registration event is written to the audit-logs table.

Figure 3.5: Customer Registration Flow

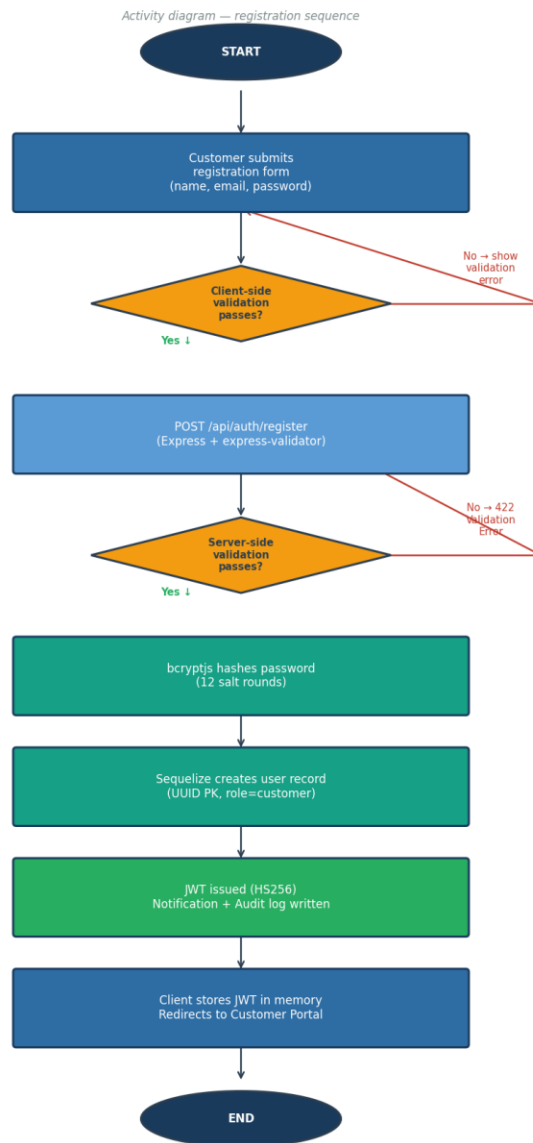


Figure 3.5. Customer Registration Flow. Activity diagram showing client-side validation, `POST /api/auth/register`, `bcrypt` hashing, user record creation, JWT issuance, and audit logging.

The M-Pesa payment flow (Figure 3.6) is the most architecturally complex process in the system, involving asynchronous communication with the Safaricom Daraja API. The flow begins when a customer selects a plan and initiates payment. The backend normalises the phone number to 254XXXXXXXXX format and creates a pending payment record. The backend then requests an OAuth access token from the Daraja API and submits an STK Push request. The Daraja API delivers a PIN prompt to the subscriber's handset. When the subscriber approves the payment, M-Pesa processes the transaction and dispatches a callback HTTP POST to the system's registered callback URL. The callback handler parses the `ResultCode` and, on success, extracts the M-Pesa receipt number, updates the payment record to completed, and calls the subscription activation service. On failure, the payment record is updated to failed and the subscription remains pending.

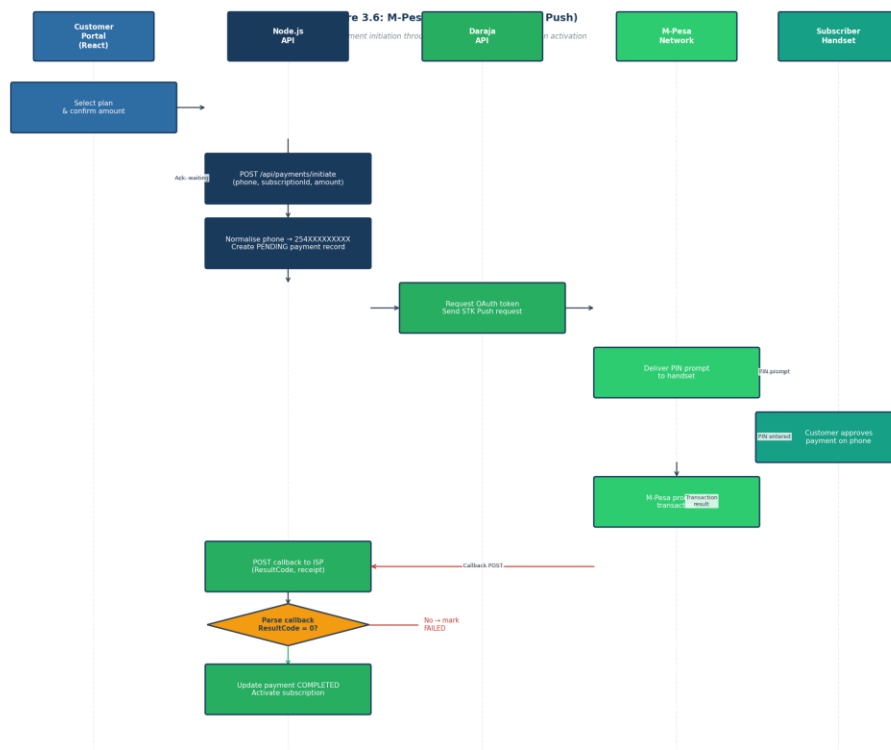


Figure 3.6. M-Pesa Payment Flow (STK Push). Swimlane diagram across Customer Portal, Node.js API, Daraja API, M-Pesa Network, and Subscriber Handset, showing the full asynchronous payment lifecycle.

The billing and invoice generation flow (Figure 3.7) can be triggered either manually by an administrator or by a bulk endpoint called by an external cron job. The service layer queries eligible subscriptions, retrieves the associated data plan pricing, and computes the base invoice amount. The data-usage table is queried to calculate total data consumed; if consumption exceeds the plan quota, an overage charge is computed at the configured per-megabyte rate. A 16% VAT amount is computed on the pre-tax subtotal. The invoice record is written with status pending and a due date 30 days from the issue date, and individual line items are written to the invoice-items table. PDFKit generates a formatted invoice PDF, and an email notification is dispatched to the subscriber.

Figure 3.7: Billing / Invoice Generation Flow

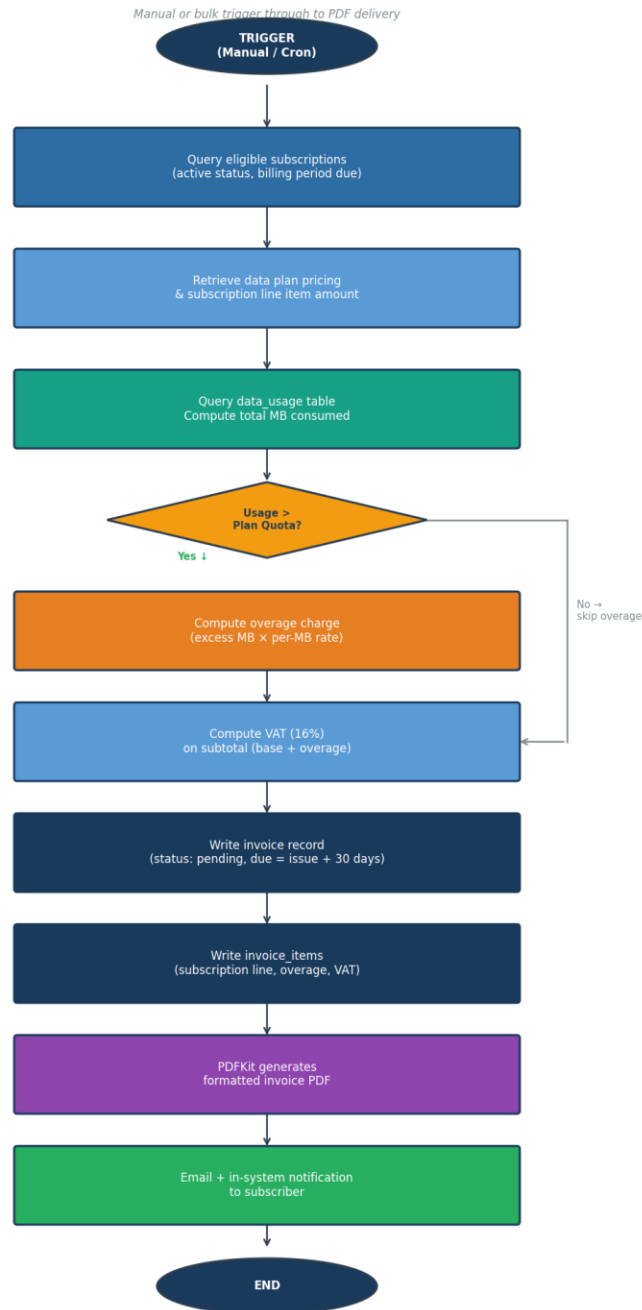


Figure 3.7. Billing and Invoice Generation Flow. Activity diagram from trigger through subscription query, usage computation, overage and VAT calculation, invoice record creation, PDF generation, and subscriber notification.

The MLR revenue prediction flow (Figure 3.8) is initiated when an administrator triggers a revenue forecast from the AI dashboard. The Node.js API dispatches a POST request to the Flask AI microservice. The Flask service queries MySQL to retrieve historical monthly revenue data, active subscriber counts, average data usage, and payment delay statistics. If the training dataset contains at least six historical observations, the service constructs the feature matrix and target vector and invokes `np.linalg.lstsq` to solve the OLS normal equations, yielding the coefficient vector. Coefficients are serialised to a JSON file for persistence. The service then applies the trained model to current period features to generate a forecast, computes feature importance as normalised absolute coefficient magnitudes, and returns results to the Node.js API, which persists the result to ai-insights and returns it to the React dashboard.

Figure 3.8: MLR Revenue Prediction Flow

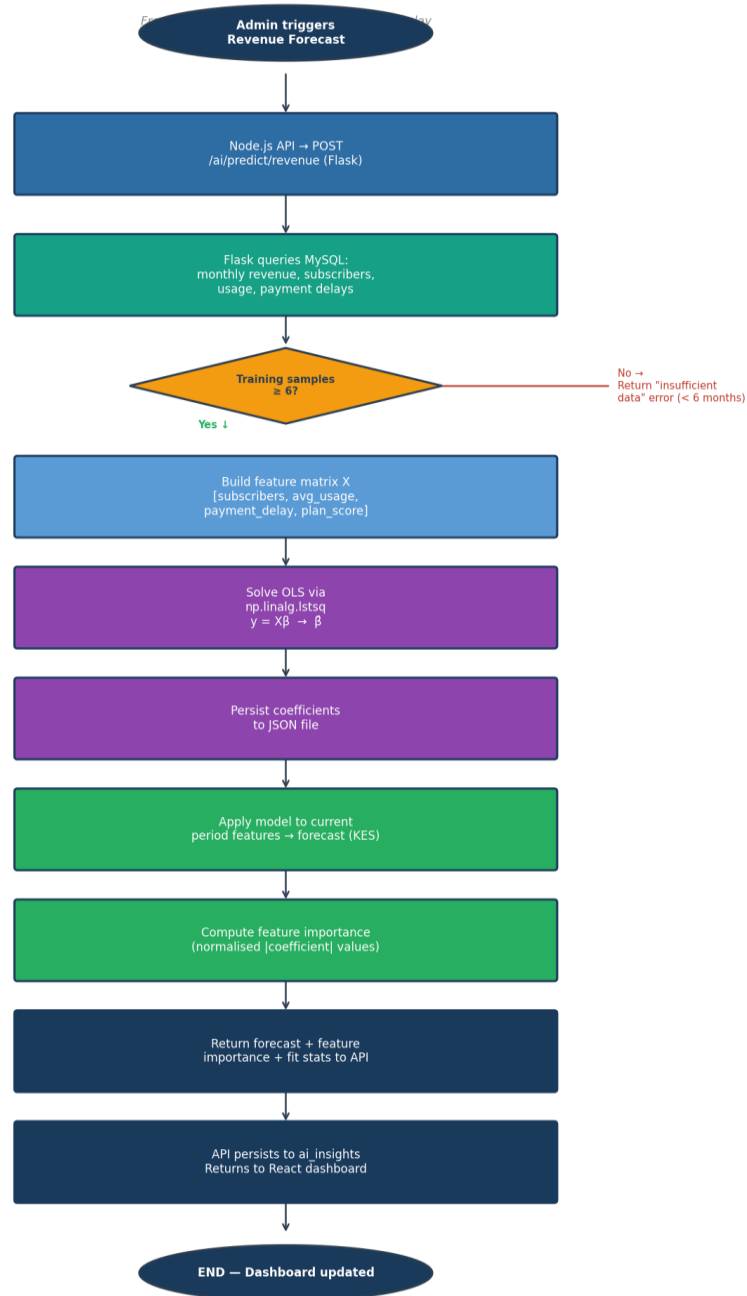


Figure 3.8. MLR Revenue Prediction Flow. Activity diagram from admin trigger through Flask microservice query, OLS coefficient estimation via `np.linalg.lstsq`, JSON persistence, forecast computation, and dashboard display.

The support ticket lifecycle (Figure 3.9) begins when a customer submits a new ticket via the customer portal. The backend creates a ticket record in the support tickets table with status open. A support staff member reviews the ticket, assigns it to themselves (changing status to in progress), and adds response notes as the investigation proceeds. When a resolution is reached, the ticket status is updated to resolved and the customer receives a notification. The customer may confirm satisfaction, triggering a final transition to closed, or may reopen the ticket if the resolution is inadequate. All ticket status transitions and responses are recorded in the audit-logs table.

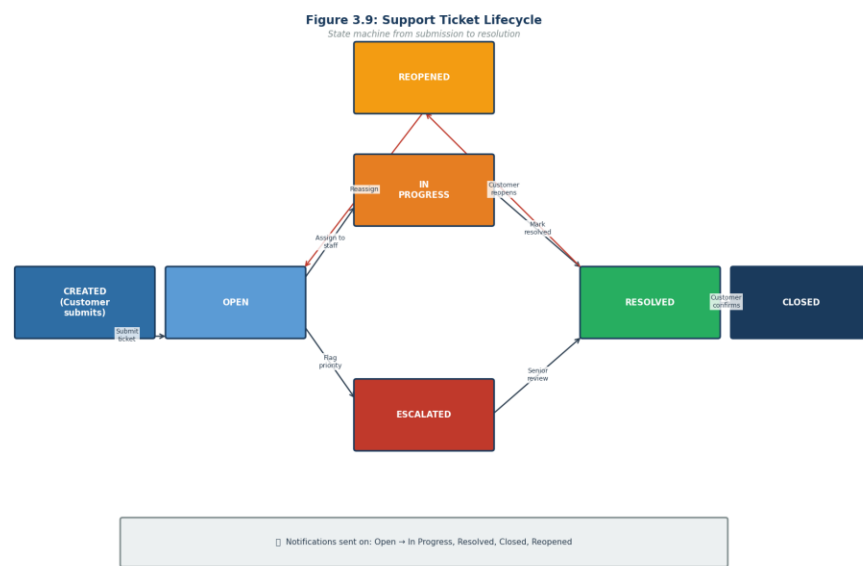


Figure 3.9. Support Ticket Lifecycle. State machine diagram showing ticket states (Created, Open, In Progress, Escalated, Resolved, Closed, Reopened) and valid transition paths.

3.8.3 Design for Non-Functional Requirements

The security design of the system implements defence in depth through multiple independent layers. The outermost layer is the Nginx reverse proxy in production, which terminates TLS/SSL and forwards only HTTPS requests to the Node.js API. The Express application layer applies the Helmet middleware as the first handler in the middleware chain. The CORS middleware restricts cross-origin requests to the configured allowlist, rejecting requests from unauthorised origins with a 403 response. Express-rate-limit caps request rates per IP address for sensitive endpoints, providing protection against credential-stuffing and brute-force attacks. Express-validator

validates and sanitises all user-supplied input, rejecting malformed requests with 422 validation error responses. The JWT verification middleware intercepts every protected route, validates the token signature and expiry, and attaches the decoded user identity to the request object. The RBAC middleware then checks the user's role claim against the required role for the requested endpoint. The Sequelize ORM exclusively uses parameterised queries for all database access, providing comprehensive SQL injection protection.

The error handling strategy employs a centralised Express error-handling middleware that intercepts all errors propagated via `next(error)` from any route handler or service function. The middleware classifies errors by type: Sequelize validation errors receive 422 responses with field-specific error details; Sequelize unique constraint errors receive 409 conflict responses; JWT errors receive 401 responses; and all unhandled errors receive generic 500 responses in production. Winston structured logging captures the full error details including stack trace to the server-side log regardless of environment, ensuring that production incidents can be fully diagnosed from logs without client-visible information disclosure. The Flask AI service implements global exception handlers that return structured JSON error responses for all failure modes, preventing raw Python tracebacks from reaching the client.

3.9 Physical Design

3.9.1 Database Design

MySQL 8 was selected as the relational database management system based on its proven reliability, strong transactional support (InnoDB engine), extensive deployment experience in the Kenyan hosting environment, and seamless integration with Sequelize ORM. All tables use UUID (universally unique identifier) values as primary keys, generated by the application layer. This choice prevents primary key exposure from revealing the total number of records in a table and supports future distributed extensions. The Sequelize migration system manages all schema changes through versioned migration files. The Entity-Relationship Diagram illustrating all entities and their relationships is presented in Figure 3.10.

The database schema comprises twelve primary entities. The users table records all system users with fields for UUID primary key, first name, last name, email (unique), hashed password, role enum (customer, admin, support), account status, and timestamps. The paranoid mode of

Sequelize adds a `deletedAt` column for soft deletes, meaning that user records are never physically removed from the database. The `data_plans` table records available subscription plans with fields for plan name, description, data allowance (in MB), validity period (days), price (decimal in KES), plan status, and a JSON features column for extensible plan metadata. The `subscriptions` table links users to data plans and tracks the subscription lifecycle with fields for user ID (FK), plan ID (FK), status enum (pending/active/expired/suspended/cancelled), start date, end date, and data consumed (MB).

The `payments` table records all payment transactions with fields for user ID (FK), subscription ID (FK), amount, payment method (mpesa), status (pending/completed/failed), M-Pesa phone number, checkout request ID from the STK Push response, M-Pesa receipt number from the callback, and timestamps. The `invoices` table stores billing documents per subscription with fields for subscription ID (FK), user ID (FK), issue date, due date, status (pending/paid/overdue/cancelled), subtotal, VAT amount, and total. The `invoice_items` table stores individual line items within an invoice with fields for invoice ID (FK), description, quantity, unit price, and line total. The `data_usage` table records per-session data consumption events with fields for subscription ID (FK), user ID (FK), bytes uploaded, bytes downloaded, session start, and session end. The `support_tickets` table stores customer service requests with fields for user ID (FK), title, description, category, status enum (open/in_progress/resolved/closed), priority, and assigned staff ID (FK to users). The `notifications`, `audit_logs`, `settings`, and `ai_insights` tables complete the schema, with the `ai_insights` table using a JSON payload column to store variable-structure ML model outputs.

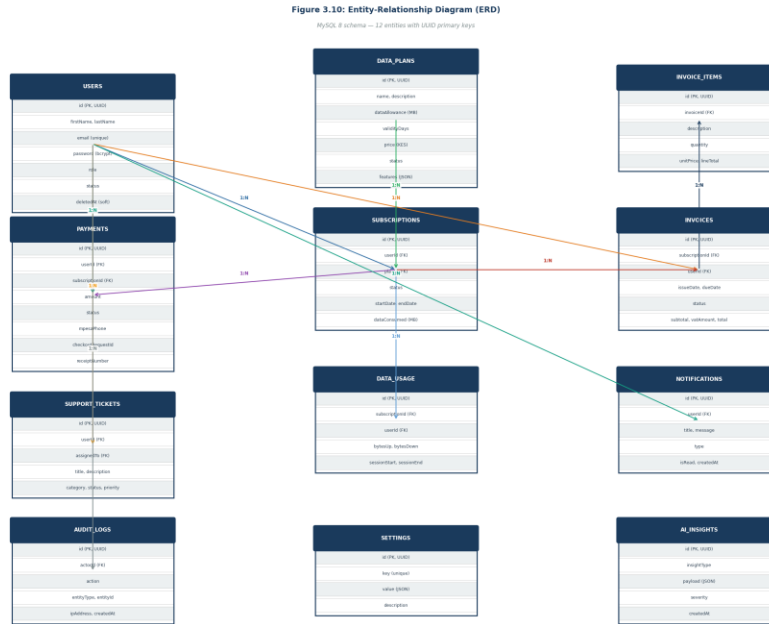


Figure 3.10. Entity-Relationship Diagram (ERD). MySQL 8 schema 12 entities with UUID primary keys; key relationships include User 1:N Subscriptions/Payments/Invoices/DataUsage/SupportTickets, DataPlan 1:N Subscriptions, Subscription 1:N Invoices/Payments/DataUsage, Invoice 1:N InvoiceItems.

3.9.2 User Interface Design

The user interface is built with Material UI 7.2, which provides a comprehensive component library implementing the Material Design specification. MUI's responsive grid system, pre-built form components, data table components, and navigation elements provide a consistent, accessible interface across device sizes. Tailwind CSS utility classes supplement MUI for custom layout spacing and component composition where MUI's default styles would require verbose theme customisation. The interface architecture is structured around a role-based layout shell: upon authentication, the application reads the user's role from the JWT payload and renders a role-appropriate navigation sidebar and route set, ensuring that administrators, support staff, and customers see only the navigation items relevant to their permissions. Sample UI wireframes for the four primary interface views are presented in Figure 3.11.

The administrator dashboard presents four KPI cards at the top of the page total monthly revenue (KES), active subscriber count, churn rate (percentage), and active anomaly count, each with a

colour-coded trend indicator. Below the KPI row, a full-width Recharts area chart displays monthly revenue over the trailing 12 months, followed by a bar chart showing new subscriber acquisition per month. The AI dashboard section presents the churn risk table ranking active subscribers by logistic regression churn probability, anomaly alert cards with severity levels, a Recharts line chart of the MLR revenue forecast for the next three months, and a bar chart of MLR feature importance scores. The customer self-service portal presents a plan browser as the primary landing page, a subscription status card with a data usage progress gauge, invoice list, M-Pesa payment modal, and the persistent AI Chat-Widget floating action button. The support ticket interface provides a tabular view of all tickets with status badges, and the AI analytics dashboard consolidates all ML outputs for administrator review.



Figure 3.11. Sample UI Wireframes Material UI 7.2 role-based responsive interface. Panels: (a) Admin Dashboard with KPI cards and Recharts charts; (b) Customer Self-Service Portal with plan browser, usage gauge, and M-Pesa payment; (c) Support Ticket Interface; (d) AI Analytics Dashboard with MLR feature importance bars and churn risk scores.

APPENDICES

Appendix A: References

- Ajzen, I. (2020). The theory of planned behavior: Frequently asked questions. *Human Behavior and Emerging Technologies*, 2(4), 314–324. <https://doi.org/10.1002/hbe2.195>
- Alalwan, A. A., Dwivedi, Y. K., Rana, N. P., & Williams, M. D. (2020). Consumer adoption of mobile payment systems: A meta-analytical review. *Information Systems Frontiers*, 22(3), 555–573. <https://doi.org/10.1007/s10796-018-9875-4>
- Gikandi, J. W., & Bloor, C. (2021). Adoption and use of mobile money services in developing countries: Evidence from Kenya. *Journal of African Business*, 22(3), 356–374. <https://doi.org/10.1080/15228916.2020.1838474>
- Khan, S., Ali, A., & Khan, F. (2022). Design and implementation of automated billing systems for internet service providers. *International Journal of Computer Applications*, 174(22), 1–8.
- Kshetri, N. (2021). The economics of mobile money in developing countries. *Telecommunications Policy*, 45(2), 102–115. <https://doi.org/10.1016/j.telpol.2020.102102>
- Laranjo, L., Dunn, A. G., Tong, H. L., Kocaballi, A. B., Chen, J., Bashir, R., ... Coiera, E. (2018). Conversational agents in healthcare: A systematic review. *Journal of the American Medical Informatics Association*, 25(9), 1248–1258. <https://doi.org/10.1093/jamia/ocy072>
- Mwangi, I., & Kimani, S. (2023). Secure API integration strategies for mobile money platforms. *African Journal of Information Systems*, 15(2), 45–60.
- Safaricom PLC. (2024). *Daraja API documentation*. Safaricom Developer Portal. <https://developer.safaricom.co.ke/>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.
- Xu, A., Liu, Z., Guo, Y., Sinha, V., & Akkiraju, R. (2020). A new chatbot for customer service on social media. *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, 3506–3510. <https://doi.org/10.1145/3025453.3025496>

Appendix B: Project Schedule

The project will be conducted over a period of three months in line with the university’s final year project timeline. The schedule is divided into key phases to ensure systematic progression and timely completion of the project.

Table B.2 Project Schedule

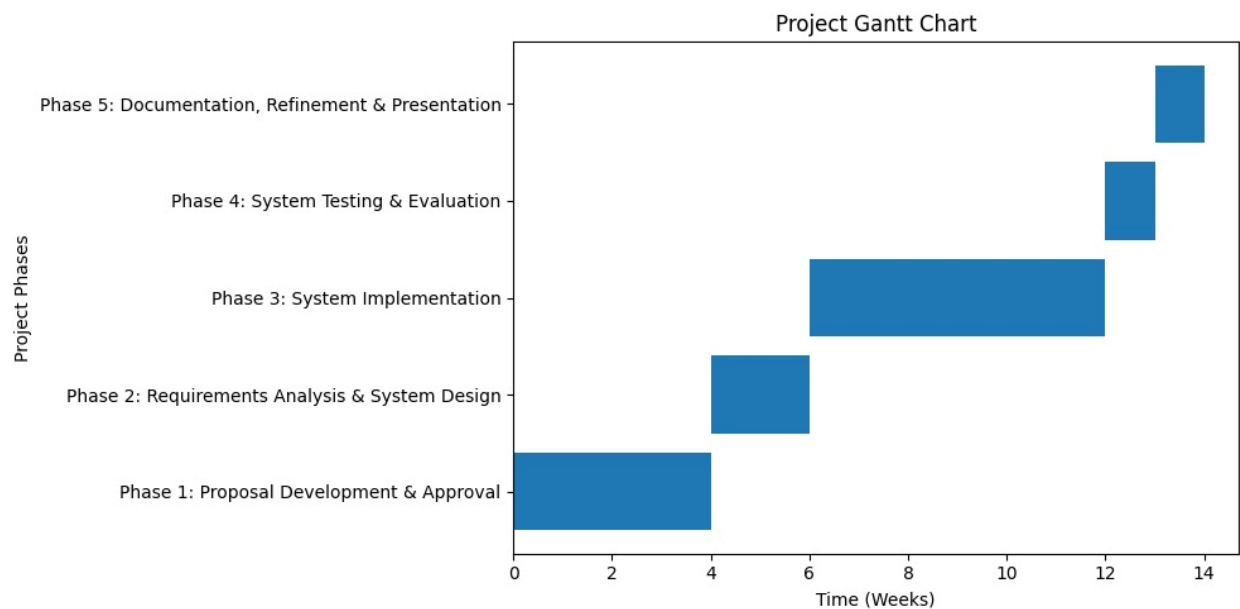


Figure D1: The diagram illustrates a grant chart for the project schedule

Appendix C: Project Budget

The estimated budget for the project is outlined below. The project primarily relies on open-source technologies to minimize costs.

ITEM	DESCRIPTION	Estimated Cost
Development Laptop & Utilities	Pro-rated academic usage cost	0
Backup Storage	Data protection & system backups	1000
Domain Registration	System hosting domain	Ksh.1000
Cloud Hosting & Testing	Backend & Database deployment	Ksh.3000
AI API Usage	Token usage & testing	Ksh.2000
M-Pesa Daraja Integration	SMS callbacks, testing costs	Ksh.1500
Email Services	Invoice and notification delivery	Ksh.2500
Performance & Load Testing	Concurrent user simulation tools	Ksh.1500
Presentation Materials	Slides, demo preparation	Ksh.500
Miscellaneous & Contingency	Unexpected expenses	Ksh.3000
Total		Ksh.16000

Table B.2 Project Budget Estimate

Appendix D: Project Resources

The successful completion of this project will require both hardware and software resources. Hardware resources include a personal computer with sufficient processing power and memory to run development tools, databases, and containerized services.

CATEGORY	RESOURCE	PURPOSE
Hardware Resources	Development Laptop/PC	System development, coding, testing, and deployment
	External Storage / Cloud Backup	Data backup and recovery
	Stable Internet Connection	API integration, cloud deployment, version control
Software Resources	Node.js & Express.js	Backend development and API creation
	React.js	Frontend development (User Interface)
	MySQL	Database management system
	Docker	Containerization and deployment consistency
	Git & GitHub	Version control and source code management
	Visual Studio Code	Code editor and development environment
AI & Analytical Tools	Multiple Linear Regression Model	Revenue prediction and anomaly detection
	LLaMA-based LLM API	Natural language support and decision assistance
Third-Party APIs & Services	M-Pesa Daraja API	Payment processing and transaction validation
	Email Service (SMTP/API)	Invoice and notification delivery
Testing & Security Tools	Postman	API testing
	Load Testing Tools	Performance evaluation
	JWT Authentication	Secure access control
Human Resources	Student Researcher	System design, development, and documentation
	Project Supervisor	Academic guidance and evaluation
	Test Users	User acceptance testing

Appendix E: System Diagrams

This appendix outlines the key system diagrams required to support the design and implementation of the proposed ISP Billing and Management System. All diagrams should be clearly labeled and captioned according to APA standards and referenced within the main report.

Figure D.2 System Architecture Diagram

This diagram illustrates the high-level architecture of the system, showing the interaction between the frontend (React.js), backend server (Node.js and Express), database (MySQL), third-party services (M-Pesa Daraja API, Email services), and the AI service layer. The diagram demonstrates data flow and integration points across system components.

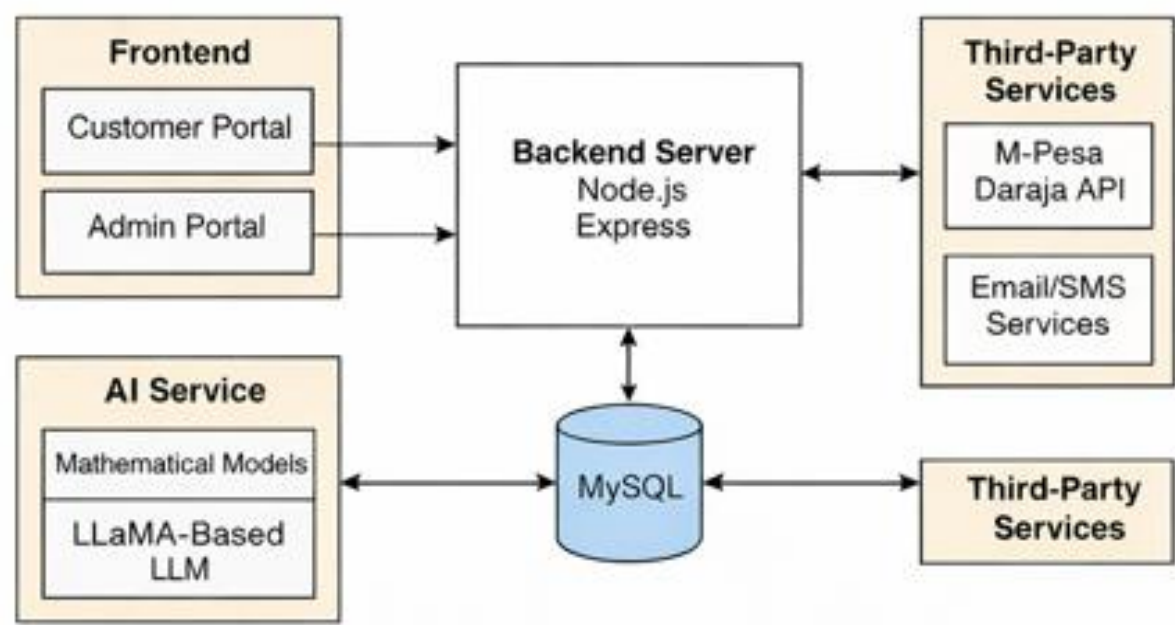


Figure D2: The diagram illustrates the component-based architecture of the ISP billing and management system.

Figure D.3 Use Case Diagram

The use case diagram represents the functional interactions between system actors and the system. The primary actors include Administrator, Support Staff, and Customer. Key use cases include user registration, subscription management, invoice generation, payment processing, viewing usage statistics, and AI-assisted support.

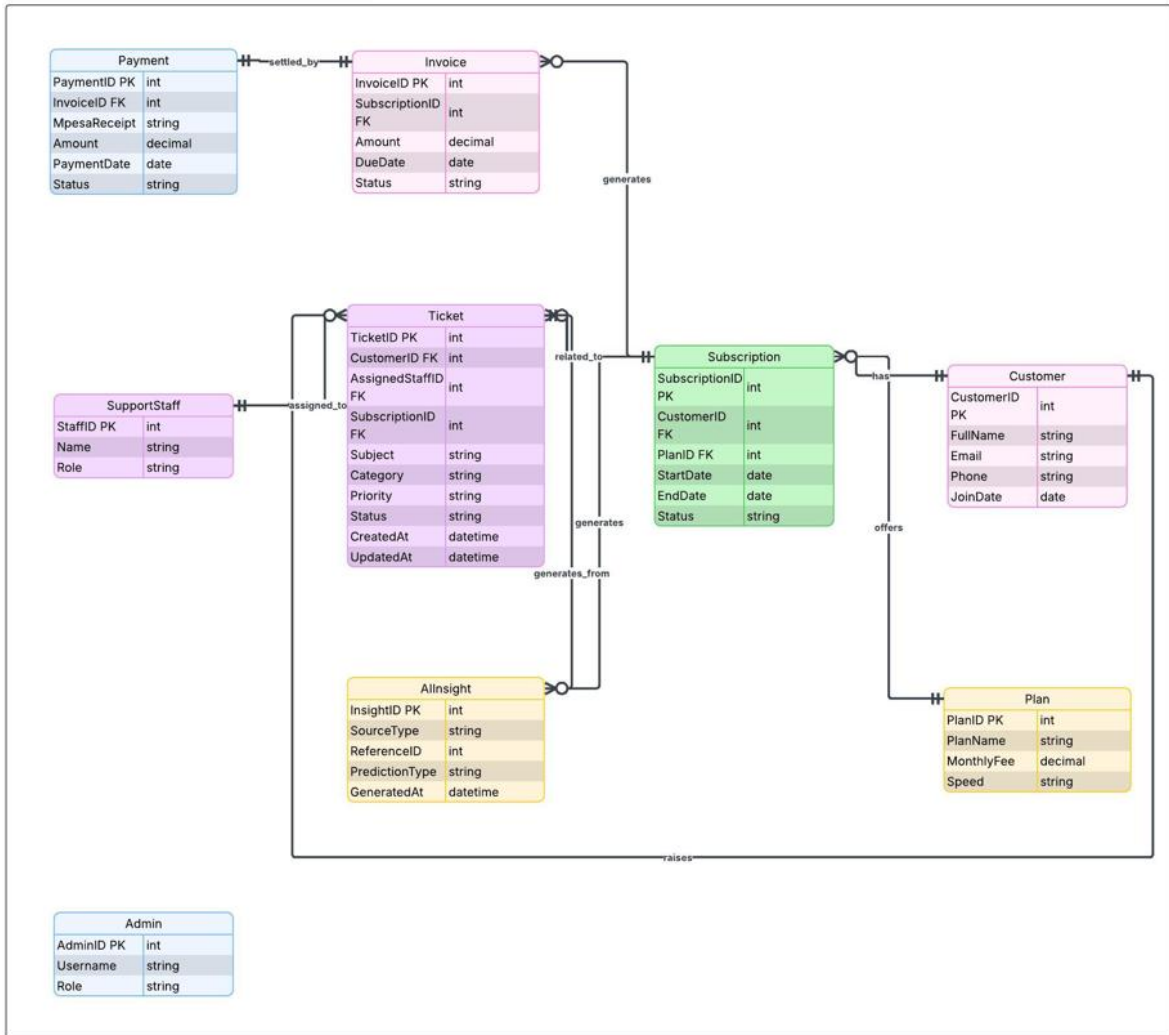


Figure D4: The ERD shows the key database entities and their relationships in the ISP billing and management system.

Figure D.5 Data Flow Diagram

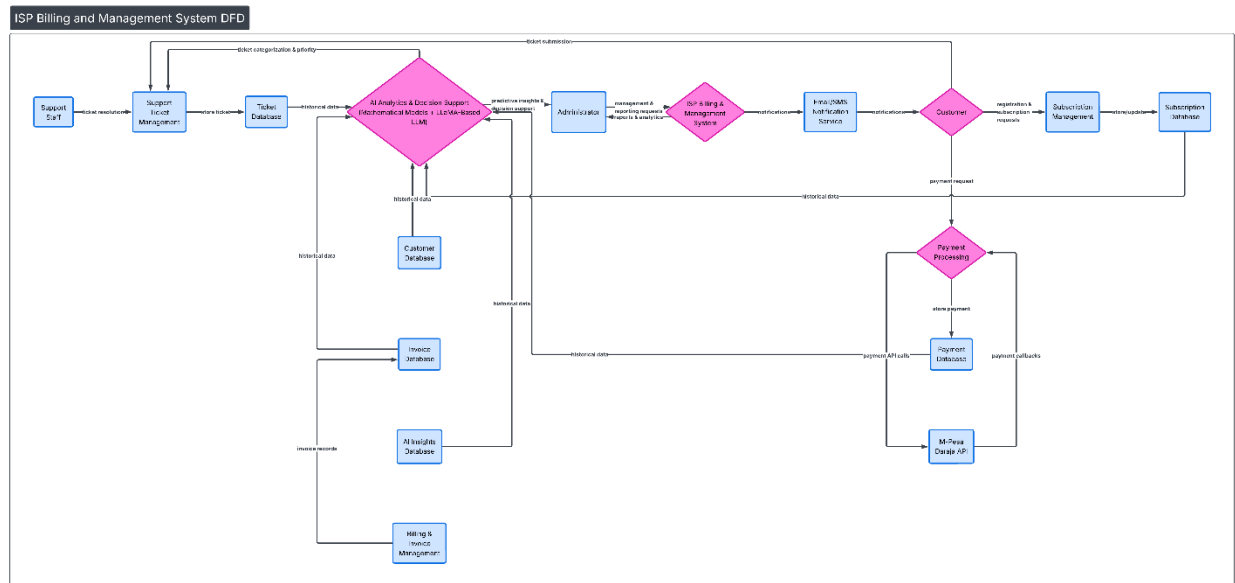


Figure D5: The DFD illustrates the data flows in the ISP's billing and management system.

These diagrams collectively provide a clear visual representation of the system's structure, behavior, and data organization, supporting both implementation and evaluation of the proposed solution.